

CHAPTER 9

Shrinkage Methods

9.1 Principal Components

Recall that if the X -matrix is orthogonal then testing and interpretation are greatly simplified. One motivation for principal components (PCs) is to rotate the X to orthogonality. For example, consider the case with two predictors depicted in Figure 9.1.

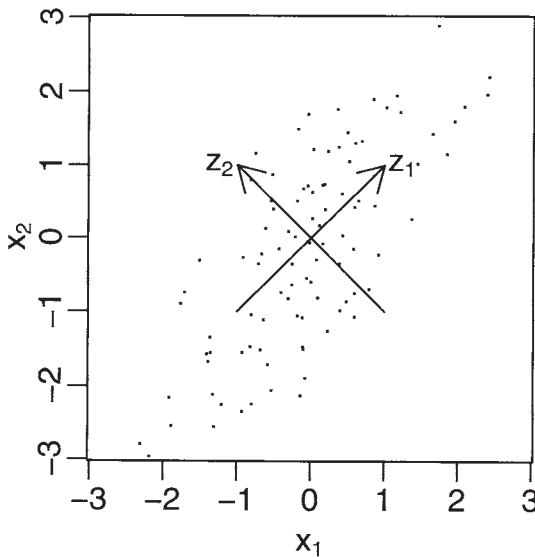


Figure 9.1 *Original predictors are x_1 and x_2 ; principal components are z_1 and z_2 .*

The original predictors, x_1 and x_2 , are clearly correlated and so the X -matrix will not be orthogonal. This will complicate the interpretation of the effects of x_1 and x_2 on the response. Suppose we rotate the coordinate axes so that in the new system, the predictors are orthogonal. Furthermore, suppose we make the rotation so that the first axis lies in the direction of the greatest variation in the data, the second in the second greatest direction of variation in those dimensions remaining and so on. These rotated directions, z_1 and z_2 in our two predictor example, are simply linear combinations of the original predictors. This is the geometrical description of PCs. We now indicate how these directions may be calculated.

We wish to find a rotation $p \times p$ matrix U such that $Z = XU$ and $Z^T Z = \text{diag}(\lambda_1, \dots, \lambda_p)$ and $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$. Zero eigenvalues indicate nonidentifiability. Since:

$$Z^T Z = X^T X^T X U$$

the eigenvalues of $X^T X$ are $\lambda_1, \dots, \lambda_p$ and the eigenvectors of $X^T X$ are the columns of U . The columns of Z are called the PCs and these are orthogonal to each other. λ_i is the variance of Z_i .

Another way of looking at it is to try to find the linear combinations of X that have the maximum variation. We find the u_1 such that $\text{var}(u_1^T X)$ is maximized subject to $u_1^T u_1 = 1$. Now find u_2 such that $\text{var}(u_2^T X)$ is maximized subject to $u_1^T u_2 = 0$ and $u_2^T u_2 = 1$. We keep finding directions of greatest variation orthogonal to those directions we have already found. A simpler version of this is illustrated in Figure 9.1.

There are some variations on this theme:

1. We can use an X that includes the intercept as a column of ones or we can center each variable by its mean in which case we dispense with the intercept since the centered variables are orthogonal to it. The latter choice is more common since it means the PCs will just be linear combinations of predictors without a constant, which is easier to interpret.
2. As well as centering each variable, we could also divide by its standard deviation, thus standardizing each variable. This is equivalent to doing principal components analysis (PCA) on the correlation matrix of the predictors. This option makes the most sense when the predictors are measured on different scales, say millimeters and grams. If no scaling is done, the PCs will be dominated by the numerically largest predictors, but this may not make sense. When all the predictors are on the same scale, say distances between locations on the face, we might choose to center only.

PCA is a general technique for reducing the dimension of data. PCA is not specifically designed for regression and most applications lie in other fields. See Johnson and Wichern (2002) for an introduction. Typically, only a few eigenvalues will be large so that almost all the variation in X will be representable by the first few PCs. Various methods are used to determine how many components should be used. In application to regression, we replace the regression $y \sim X$ with $y \sim Z$ where we use only a few columns of Z . This is known as *principal components regression* or PCR. The technique is used in two distinct ways.

Explanation When the goal of the regression is to find simple, well-fitting and understandable models for the response, PCR *may* help. The PCs are linear combinations of the predictors. If we use these directly, then little is gained, but perhaps a close approximation may have a good interpretation. For example, suppose:

$$Z_1 = 0.71X_1 + 0.02X_2 - 0.06X_3 - 0.69X_4$$

Now Z_1 is approximately proportional to $X_1 - X_4$. So, suppose X_1 was a person's height and X_4 was the distance between his or her hands when outstretched to the side, this would represent a measure of the shape of the person. However, if X_4 was the person's weight, then $X_1 - X_4$ would be harder to interpret. If this interpretation process works well, a large number of X is reduced to a much smaller number of (approximate) Z that can be understood and used to model the response. However, for this to work, we typically need the predictors to measure quantities for which linear combinations are interpretable—usually the predictors would need the same units. Furthermore, we would need some luck to get interpretable PCs and we would need to make some creative approximations. These requirements severely restrict the utility of PCR for explanatory purposes.

Prediction It may be that we can make better predictions with a small number of Z than a much larger number of X . We shall see this in the example that follows. Success requires that we make a good choice of the number of components. PCA does not use y so it is possible (although less likely) that some lesser PC is actually very important in predicting the response.

A Tecator Infratec Food and Feed Analyzer working in the wavelength range of 850 to 1050 nm by the near infrared transmission (NIT) principle was used to collect data on samples of finely chopped pure meat. 215 samples were measured. For each sample, the fat content was measured along with a 100 channel spectrum of absorbances. Since determining the fat content via analytical chemistry is time consuming, we would like to build a model to predict the fat content of new samples using the 100 absorbances which can be measured more easily. See Thodberg (1993).

The true performance of any model is hard to determine based on just the fit to the available data. We need to see how well the model does on new data not used in the construction of the model. For this reason, we will partition the data into two parts—a *training sample* consisting of the first 172 observations that will be used to build the models and a *testing sample* of the remaining 43 observations.

Let's start with the least squares fit:

```
> data (meatspec)
> model 1 <- lm (fat ~ ., meatspec[1:172,])
> summary (modell) $r.squared
[1] 0.99702
```

We see that the fit of this model is already very good in terms of R^2 . How well does this model do in predicting the observations in the test sample? We need a measure of performance—we use root mean square error (RMSE):

$$\sqrt{(\sum_{i=1}^n (\hat{y}_i - y_i)^2 / n)}$$

where $n=43$ in this instance. We find for the training sample that:

```
> rmse <- function (x, y) sqrt (mean ( (x-y)^2 ) )
> rmse (modell$fit, meatspec$fat [1:172])
[1] 0.69032
```

while for the test sample:

```
> rmse (predict (modell, meatspec [173:215,]), meatspec$fat
[173:215] )
[1] 3.814
```

We see that the performance is much worse for the test sample. This is not unusual, as the fit to the data we have almost always gives an overoptimistic sense of how well the model will do with future data. In this case, the actual error is about five times greater than the model itself suggests.

Now, it is quite likely that not all 100 predictors are necessary to make a good prediction. In fact, some of them might just be adding noise to the prediction and we could improve matters by eliminating some of them. We use the default stepwise model selection:

```
> model2 <- step (modell)
> rmse(model2$fit, meatspec$fat [1:172] )
[1] 0.7095
> rmse (predict (model2, meatspec [173:215,]),
meatspec$fat[173:215] )
[1] 3.5902
```

The model selection step removed 28 variables. Of course, the nominal fit got a little worse as it always will when predictors are removed, but the actual performance improved somewhat from 3.81 to 3.59.

Now let's compute the PCA on the training sample predictors:

```
> library (mva)
> meatpca <- prcomp(meatspec[1:172, -101])
```

We can examine the square roots of the eigenvalues:

```
> round (meatpca$sdev, 3)
[1] 5.055 0.511 0.282 0.168 0.038 0.025 0.014 0.011 0.005
etc
[97] 0.000 0.000 0.000 0.000
```

It is better to examine the square roots because the eigenvalues themselves are the variances of the PCs, but standard deviations are a better scale for comparison. We see that the first PC accounts for about ten times more variation than the second. The contribution drops off sharply. This suggests that most of the variation in the predictors can be explained with just a few dimensions.

The eigenvalues can be found in the object `meatpca$rotation`. We plot these vectors against the predictor number (which represents a range of frequencies in this case) in Figure 9.2 using:

```
> matplot (1:100, meatpca$rotation [,1:3] , type="l",
xlab="Frequency", ylab="")
```

These vectors are the linear combinations of the predictors that generate the PCs. We see that the first PC comes from an almost constant combination of the frequencies. It measures whether the predictors are generally large or small. The second PC represents a contrast between the higher and lower frequencies. The third is more difficult to interpret. It is sometimes possible, as in this example, to give some meaning to the PCs. This is typically a matter of intuitive interpretation. In some other cases, no interpretation can be found—this is almost always the case when the predictors measure variables on different scales (like a person's height and age).

We can get the PCs themselves from the columns of the object `meatpca$x`. Let's use the first four PCs to predict the response:

```
> model13 <- lm (fat ~ meatpca$x [,1:4], meatspec [1:172,])
> rmse(model13$fit, meatspec$fat [1:172])
[1] 4.0647
```

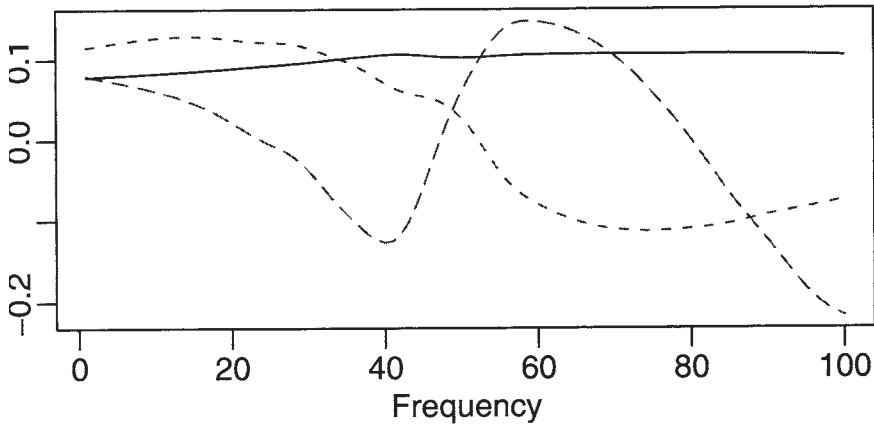


Figure 9.2 *Eigenvectors for the PCA of the meat spectrometer data. The solid line corresponds to the first PC, the dotted is for the second PC and the dashed is for the third PC.*

We do not expect as good a fit using only four variables instead of the 100. Even so, considering that, the fit is not much worse than the much bigger models.

PCR is an example of *shrinkage* estimation. Let's see where the name comes from. We plot the 100 slope coefficients for the full least squares fit:

```
> plot(model1$coef[-1], ylab="Coefficient")
```

which is shown in the left panel of Figure 9.3. We see that the coefficients range is in the thousands and that the adjacent coefficients can be very different. This is perhaps surprising because one might expect that adjacent frequencies might have a very similar effect on the response. Because the PCs represent a linear combination of the 100 predictors, we can compute the contribution of each in the PCR of just the four PCs and plot it. The PCR model is $y = Z\gamma + \epsilon$ which is $y = XU\gamma + \epsilon$. We compute $U\gamma$ and plot it in the panel on the left of Figure 9.3.

```
> svb <- meatpca$rot[,1:4] %*% model3$coef[-1]
> plot(svb, ylab="Coefficient")
```

Here we see that the coefficients are much smaller, ranging from -10 to 10 rather than in the thousands. Instead of wildly varying coefficients in the least squares case, we have a more stable result. This is why the effect is known as shrinkage. Furthermore, there is some smoothness between adjacent frequencies.

Why use four PCs here? The standard advice for choosing the number of PCs to represent the variation in X is to choose the number beyond which all the eigenvalues are relatively small. A good way to determine this number is to make a *scree plot*, which simply makes an index plot of the eigenvalues. We show here the square roots of the eigenvalues and only the first ten values to focus on the area of interest; see Figure 9.4:

```
> plot(meatpca$sdev[1:10], type="l", ylab="SD of PC",
xlab="PC number")
```

We could make a case for using only the first PC, but there is another identifiable “elbow” at five indicating the choice of four PCs. Now let’s see how well the test sample is predicted. The default version of PCs used here centers the predictors so we need to impose the same centering (using the means of the training sample) on the predictors:

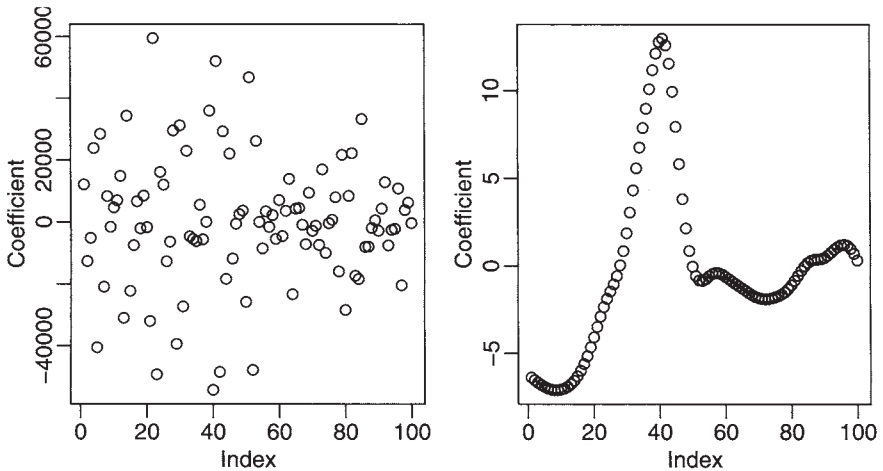


Figure 9.3 *Coefficients for the least squares fit on the left and for the PCR with four components on the right.*

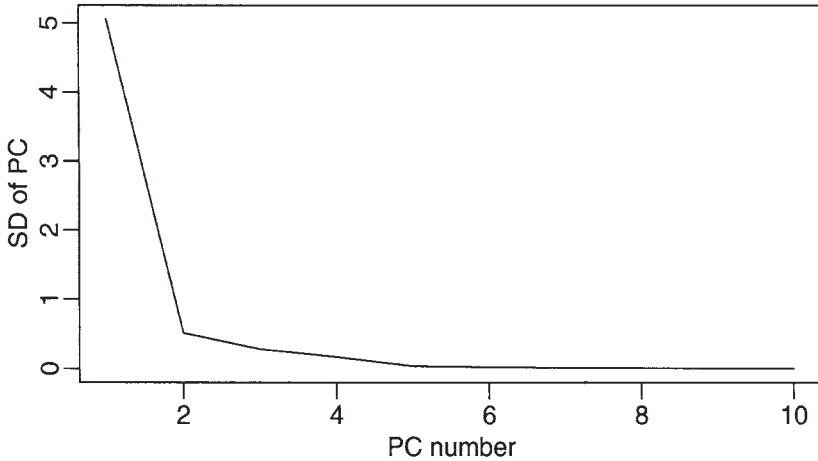


Figure 9.4 *Scree plot of the standard deviations of the first ten principal components.*

```
> mm <- apply (meatspec[1:172, -101], 2, mean)
> tx <- as.matrix (sweep (meatspec [173:215, -101], 2, mm))
```

We now form the four linear combinations determined by the first four eigenvectors and compute the predicted values:

```
> nx <- tx %*% meatpca$rot[,1:4]
> pv <- cbind (1, nx) %*% model13$coef
```

We find the RMSE to be:

```
> rmse (pv, meatspec$fat [173:215])
[1] 4.534
```

which is not at all impressive. It turns out that we can do better by using more PCs—we figure out how many would give the best result on the test sample:

```
> rmsmeat <- numeric(50)
> for (i in 1:50) {
+ nx <- tx %*% meatpca$rot[,1:i]
+ model13 <- lm (fat ~ meatpca$x[,1:i] , meatspec[1:172,])
+ pv <- cbind (1, nx) %*% model13$coef
+ rmsmeat [i] <- rmse(pv, meatspec$fat[173:215] )
+ } }
> plot (rmsmeat, ylab="Test RMS")
> which.min (rmsmeat)
```

```
[1] 27
> min (rmsmeat)
[1] 1.8549
```

The plot of the RMSE is seen in Figure 9.5. The best result occurs for 27 PCs for which the RMSE is far better than anything achieved thus far. Of course, in practice we would not have access to the test sample in advance and so we would not know to use 27 components. We could, of course, reserve part of our original dataset for testing. This is sometimes called a *validation sample*. This is a reasonable strategy, but the downside is that we lose this sample from our estimation which degrades its quality. Furthermore, there is the question of which and how many observations should go into the validation sample. We can avoid this dilemma with the use of *crossvalidation* (CV). We divide the data into m parts, equal (or close to) in size. For each part, we use the rest of the data as the training set and that part as the test set. We evaluate the criterion of interest, RMSE in this case. We repeat for each part and average the result.

The pls.pcr package can compute this CV. We center the training set X and divide the data into ten parts for the CV:

```
> library (pls.pcr)
> trainx <- as.matrix (sweep (meatspec [1:172,-101] , 2, mm))
> pcrg <- pcr(trainx,meatspec$fat[1:172] ,1:50,
  validation="CV", grpsize=10)
> plot (pcrg$validat$RMS, ylab="CV RMS", xlab="No. of components")
```

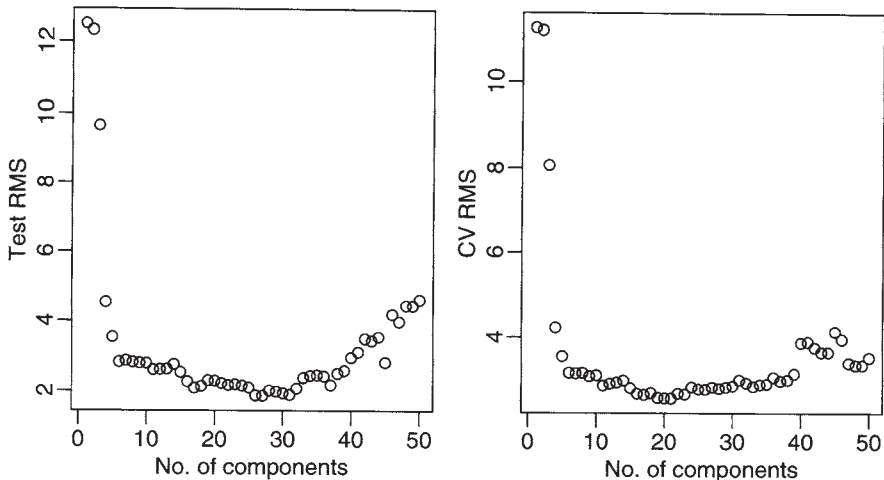


Figure 9.5 *RMS for the test sample on the left and RMS estimated using CV for varying numbers of PCs on the right.*

The crossvalidated estimates of the RMSE are shown in the right panel of Figure 9.5. The minimum occurs at 21 components. This gives an RMSE on the test sample, 2.21, that is close to the optimum.

9.2 Partial Least Squares

Partial least squares (PLS) is a method for relating a set of input variables $X_1, \dots, X_m$ and outputs $Y_1, \dots, Y_l$. PLS was developed by Herman Wold—see Wold, Ruhe, Wold, and Dunn (1984). PLS regression is comparable to PCR in that both predict the response using some number of linear combinations of the predictors. The difference is that while PCR ignores Y in determining the linear combinations, PLS regression explicitly chooses them to predict Y as well as possible.

We will consider only univariate PLS—that is to say $l=1$ so that Y is scalar. We will attempt to find models of the form:

$$\hat{y} = \beta_1 T_1 + \dots + \beta_p T_p$$

where T_k is a linear combination of the X_s . See Figure 9.6

Various algorithms have been presented for computing PLS. Most work by iteratively determining the T_i s to predict y well, but at the same time maintaining orthogonality. One criticism of PLS is that it solves no well-defined modeling problem, which makes it difficult to distinguish between the competing algorithms on theoretical rather than empirical grounds. Garthwaite (1994) presents an intuitive algorithm, but de Jong (1993) describes the SIMPLS method, which is now the most well-known. Several other algorithms exist.

As with PCR, we must choose the number of components carefully. CV can be helpful in doing this. We apply PLS to the meat spectroscopy data using CV to select the number

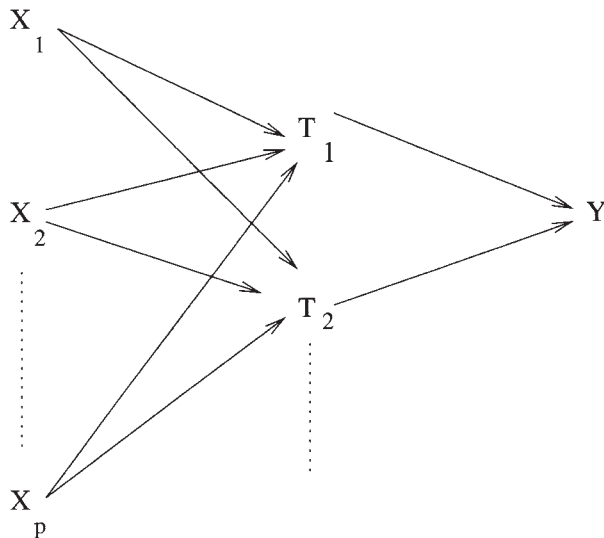


Figure 9.6 *Schematic representation of partial least squares.*

of components. We center the predictors first and compute the PLS on all models up to size 50. We plot the linear combination used for a four-component model in the left panel of Figure 9.7. The crossvalidated estimates of the RMSE are shown in the right panel.

```
> trainx <- as.matrix(sweep(meatspec[1:172, -101], 2, mm))
> plsg <- pls(trainx, meatspec$fat[1:172], 1:50,
  validation="CV", grpsize=10)
> plot(plsg$training$B[, , 4], ylab="Coefficient")
> plot(plsg$validation$RMS, ylab="CV RMS", xlab="No. of
  components")
```

We see that the effective linear combination of the predictors in the four-component model is similar to the PCR indicating the shrinkage effect. As before, four components do not appear to be enough—we need around 14 components as suggested by the crossvalidated estimate of the RMSE. We need only half the number of components as PCR, which is expected since we are using information about the response. Note that the CV randomly divides the data into ten groups so if you repeat this calculation, you will not get exactly the same result.

Now we determine the performance on the training set for the 14-component model:

```
> ypred <- plsg$training$Ypred[, , 14]
> rmse(ypred, meatspec$fat[1:172])
[1] 1.9528
```

which is similar to PCR, but now see how we do on the test set. Because the response is centered by PLS we need to add the mean response for the training set back in. We also need to center the test set:

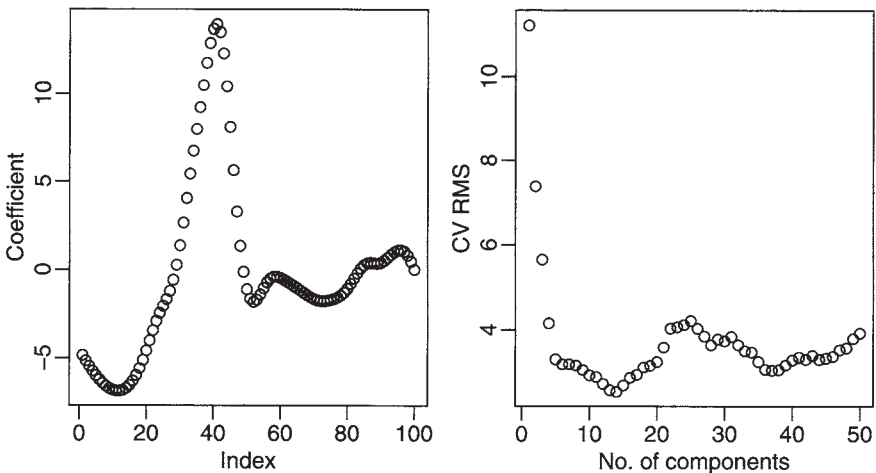


Figure 9.7 *Coefficients of X for a four-component PLS model are shown on the left. Estimated CV error is shown on the right.*

```

> testx <- as.matrix(sweep(meatspec[173:215,-101], 2, mm))
> plsbeta <- plsg$training$B[, , 14]
> ytpred <- testx %*% plsbeta + mean(meatspec$fat[1:172])
> rmse(ytpred, meatspec$f at [173:215])
[1] 2.0112

```

which is slightly better than the 2.21 achieved by PCR.

We have not checked any diagnostics in this analysis. PLS and PCR are just as sensitive to assumptions as OLS so these are still mandatory in any full analysis.

PCR and PLS compared

PCR and PLS have the biggest advantage over OLS when there are large numbers of variables relative to the number of cases. They do not even require that $n \geq p$. PCR attempts to find linear combinations of the predictors that explain most of the variation in these predictors using just a few components. The purpose is dimension reduction. Because the PCs can be linear combinations of all the predictors, the number of variables used is not always reduced. Because the PCs are selected using only the X -matrix and not the response, there is no definite guarantee that the PCR will predict the response particularly well, although it turns out that way. If it happens that we can interpret the PCs in a meaningful way, we may achieve a much simpler explanation of the response. Thus PCR is geared more towards explanation than prediction.

In contrast, PLS finds linear combinations of the predictors that best explain the response. It is most effective when there are large numbers of variables to be considered. If successful, the variability of prediction is substantially reduced. On the other hand, PLS is virtually useless for explanation purposes.

9.3 Ridge Regression

Ridge regression makes the assumption that the regression coefficients (after normalization) are not likely to be very large. The idea of shrinkage is therefore embedded in the method. It is appropriate for use when the design matrix is collinear and the usual least squares estimates of β appear to be unstable.

Suppose that the predictors have been centered by their means and scaled by their standard deviations and that the response has been centered. The ridge regression estimates of β s are then given by:

$$\hat{\beta} = (X^T X + \lambda I)^{-1} X^T y$$

The use of ridge regression can be motivated in two ways. Suppose we take a Bayesian point of view and put a prior (multivariate normal) distribution on β that expresses the belief that smaller values of β are more likely than larger ones. Large values of λ correspond to a belief that the β are really quite small whereas smaller values of λ correspond to a more relaxed belief about β . This is illustrated in Figure 9.8.

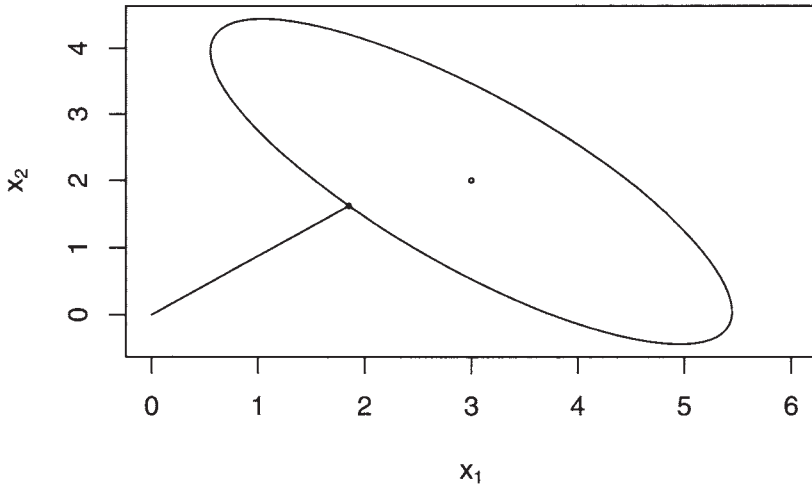


Figure 9.8 Ridge regression illustrated. The least squares estimate is at the center of the ellipse while the ridge regression is the point on the ellipse closest to the origin. The ellipse is a contour of equal density of the posterior probability, which in this case will be comparable to a confidence ellipse. λ controls the size of the ellipse—the larger λ is, the larger the ellipse will be.

Another way of looking at it is to suppose we place some upper bound on $\beta^T \beta$ and then compute the least squares estimate of β subject to this restriction. Use of Lagrange multipliers leads to ridge regression. The choice of λ corresponds to the choice of an upper bound in this formulation.

λ may be chosen by automatic methods, but it is also safer to plot the values of $\hat{\beta}$ as a function of λ . You should pick the smallest value of λ that produces stable estimates of β

We demonstrate the method on the meat spectroscopy data; $\lambda=0$ corresponds to least squares while we find that as $\lambda \rightarrow \infty$: $\hat{\beta} \rightarrow 0$

```
> library (MASS)
> yc <- meatspec$fat[1:172]-mean(meatspec$fat[1:172])
> gridge <- lm.ridge (yc ~ trainx, lambda =
seq(0,5e-8,1e-9))
> matplot (gridge$lambda, t(gridge$coef), type="l",lty=1,
xlab=expression (lambda), ylab=expression (hat (beta)))
```

Some experimentation was necessary to determine the appropriate range of λ . The *ridge trace plot* is shown in Figure 9.9.

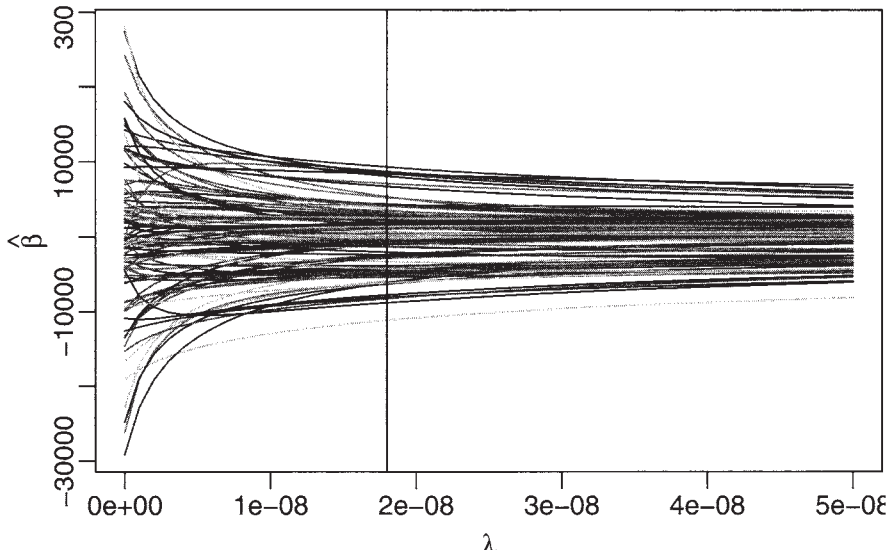


Figure 9.9 Ridge trace plot for the meat spectroscopy data. The generalized crossvalidation choice of λ is shown as a vertical line.

Various automatic selections for λ are available:

```
> select (gridge)
modified HKB estimator is 1.0583e-08
modified L-W estimator is 0.70969
smallest value of GCV at 1.8e-08
> abline (v=1.8e-8)
```

We will use the generalized crossvalidation (GCV) estimate of $1.8e^{-8}$. First, we compute the training sample performance. This ridge regression both centers and scales the predictors, so we need to do the same in computing the fit. Furthermore, we need to add back in the mean of the response because of the centering:

```
> which.min (gridge$GCV)
1.8e-08
19
> ypredg < - scale (trainx, center=FALSE,
scale=gridge$scales)
%% gridge$coef [, 19] + mean(meatspec$fat[1:172])
> rmse(ypredg,meatspec$fat[1:172])
[1] 0.80454
```

which is comparable to the above, but for the test sample we find:

```
> ytpredg < - scale (testx, center=FALSE, scale=gridge$scales)
%% gridge$coef [, 19] + mean(meatspec$fat[1:172])
> rmse(ytpredg, meatspec$fat[173:215])
[1] 4.0966
```

which is dismayingly poor. However, a closer examination of the predictions reveals that just one of the ridge predictions is bad:

```
> c (ytpredg [13], ytpred [13], meatspec$fat [172+13] )
    185      185
11.188 35.690 34.800
```

The PLS prediction (second) is close to the truth (third), but the ridge prediction is bad. If we remove this case:

```
> rmse (ytpredg[-13], meatspec$fat[173:215] [-13])
[1] 1.9765
```

we get a good result.

Ridge regression estimates of coefficients are biased. Bias is undesirable, but it is not the only consideration. The mean-squared error (MSE) can be decomposed in the following way:

$$E(\hat{\beta} - \beta)^2 = (E(\hat{\beta} - \beta))^2 + E(\hat{\beta} - E\hat{\beta})^2$$

Thus the MSE of an estimate can be represented as the square of the bias plus the variance. Sometimes a large reduction in the variance may be obtained at the price of an increase in the bias. If the MSE is reduced as a consequence, then we may be willing to accept some bias. This is the trade-off that ridge regression makes—a reduction in variance at the price of an increase in bias. This is a common dilemma.

Frank and Friedman (1993) compared PCR, PLS and ridge regression and found the best results for ridge regression. Of course, for any given dataset any of the methods may prove to be the best, so picking a winner is difficult.

Exercises

1. Using the seatpos data, perform a PCR analysis with hipcenter as the response and HtShoes, Ht, Seated, Arm, Thigh and Leg as predictors. Select an appropriate number of components and give an interpretation to those you choose. Add Age and Weight as predictors and repeat the analysis. Use both models to predict the response for predictors taking these values:

Age	Weight	HtShoes	Ht	Seated
64.800	263.700	181.080	178.560	91.440
Arm	Thigh	Leg		
35.640	40.950	38.790		

2. Fit a PLS model to the seatpos data with hipcenter as the response and all other variables as predictors. Take care to select an appropriate number of components. Use the model to predict the response at the values of the predictors specified in the first question.

3. Fit a ridge regression model to the `seatspos` data with `hipcenter` as the response and all other variables as predictors. Take care to select an appropriate amount of shrinkage. Use the model to predict the response at the values of the predictors specified in the first question.
4. Take the `bodyfat` data, and use the percentage of body fat as the response and the other variables as potential predictors. Remove every tenth observation from the data for use as a test sample. Use the remaining data as a training sample building the following models:
 - (a) Linear regression with all predictors
 - (b) Linear regression with variables selected using AIC
 - (c) Principal component regression
 - (d) Partial least squares
 - (e) Ridge regression

Use the models you find to predict the response in the test sample. Make a report on the performance of the models.